

BAHRIA UNIVERSITY, ISLAMABAD

Department of Computer Science & Information Technology

BS Information Technology — 7th Semester

Database Administration and Management (DAM / DBAM) — Spring 2026

AI Database Analyzer *Comprehensive Project Analysis Report*

Project Description

An enterprise-grade, AI-driven database administration and architecture tool that automates comprehensive schema analysis and optimization audits via hybrid AI integration (Google Gemini Cloud + Ollama Local), supporting SQL Server, PostgreSQL, and MySQL across .NET MAUI and Flutter platforms.

Course: Database Administration & Management (DAM)

Semester: Spring 2026

Date: May 17, 2026

Name: M.Talha

Enorllments: 01-135231-070

Shams-ul-Islam

01-135231-067

Contents

1	Executive Summary	4
1.1	Key Achievements	4
2	Project Overview	5
2.1	Vision & Objectives	5
2.2	Problem Statement	5
2.3	Solution Approach	5
3	System Architecture	7
3.1	High-Level Architecture Overview	7
3.2	Layered Architecture	7
3.2.1	Layer Descriptions	8
4	Technology Stack	9
4.1	Backend & Core Services	9
4.2	Database Connectivity	9
4.3	Frontend — Desktop (MAUI)	9
4.4	Frontend — Mobile/Web (Flutter)	10
4.5	AI Integration	10
4.6	Reporting & Visualization	10
4.7	Core Dependency Graph	10
5	Data Flow Analysis	11
5.1	Connection & Schema Extraction Flow	11
5.2	Analysis & Report Generation Flow	12
6	Use Case Diagrams	14
6.1	Primary User — Database Administrator	14
6.2	Extended Use Cases — Analysis Scenarios	14
7	Component Architecture	15
7.1	MAUI Application Component Diagram	15
7.2	Flutter Application Component Diagram	16
8	Key Features & Capabilities	18
8.1	Multi-Database Connectivity	18
8.2	Hybrid AI Integration	18
8.2.1	Google Gemini (Cloud)	18
8.2.2	Ollama (Local)	18

8.2.3	Smart Toggling	19
8.3	Professional Reporting	19
8.4	Interactive Schema Visualization	19
8.5	Expert-Level Analysis Prompts	20
8.5.1	Performance & Indexing (7 Analyses)	20
8.5.2	Schema Design & Data Modeling (13 Analyses)	20
8.5.3	Security & Privacy (8 Analyses)	20
8.5.4	Scalability & Architecture (3+ Analyses)	21
8.6	Customizable Analysis	21
9	Database Support	22
9.1	SQL Server	22
9.2	PostgreSQL	22
9.3	MySQL	22
10	Implementation Details	24
10.1	State Management Architecture	24
10.1.1	MAUI — MVVM Toolkit	24
10.1.2	Flutter — Provider Pattern	24
10.2	Dependency Injection Setup	24
10.3	Error Handling & Validation	25
10.4	Performance Optimizations	25
11	User Interface & Workflows	26
11.1	MAUI Desktop Application — Step-by-Step Workflow	26
11.1.1	Visual Theme	26
11.2	Flutter Mobile/Web Application	26
12	Analysis Domains	28
12.1	Performance & Indexing Analysis	28
12.2	Schema Design & Data Modeling	28
12.3	Security & Compliance	29
12.4	Scalability & Architecture	29
13	Security & Data Handling	31
13.1	Credential Management	31
13.2	Data Privacy	31
13.3	Access Controls	31
14	Deployment Architecture	33
14.1	MAUI Desktop Deployment	33
14.2	Flutter Application Deployment	33
14.3	Infrastructure Requirements	34
15	Testing & Validation	35
15.1	Unit Testing Strategy	35
15.2	Integration Testing	35
15.3	Manual Testing Checklist	36
15.3.1	Connection Phase	36

15.3.2	Extraction Phase	36
15.3.3	Analysis Phase	36
15.3.4	Report Phase	36
15.3.5	Settings Phase	36
16	Project Metrics	38
16.1	Codebase Statistics	38
16.2	Features Coverage	38
17	Architecture Decision Records (ADRs)	39
17.1	ADR-001: Dual-Platform Strategy (MAUI + Flutter)	39
17.2	ADR-002: Hybrid AI Architecture	39
17.3	ADR-003: MVVM for State Management	39
18	Conclusion	41
18.1	Project Achievements	41
18.2	Innovation Highlights	41
18.3	Educational Value (DAM Course)	41
18.4	Future Enhancement Opportunities	42
	References	43
A	Sample Configuration Files	44
A.1	MAUI Dependency Injection Configuration	44
A.2	Flutter pubspec.yaml	44

1. Executive Summary

AI Database Analyzer is an enterprise-grade, AI-driven database administration and architecture tool designed to automate comprehensive schema analysis and optimization audits. The system bridges the gap between raw database metadata and actionable insights by leveraging hybrid AI models (Google Gemini Cloud and Ollama Local processing).

1.1 Key Achievements

- ✓ **Dual-Platform:** .NET MAUI (desktop) + Flutter (cross-platform mobile/web)
- ✓ **Hybrid AI Integration:** Seamless toggling between cloud (Gemini) and local (Ollama) processing
- ✓ **Multi-Database Support:** SQL Server, PostgreSQL, and MySQL
- ✓ **30+ Specialized Audit Prompts:** Covering performance, security, scalability, and design
- ✓ **MVVM Architecture:** Enterprise-grade clean architecture patterns
- ✓ **Professional PDF Reporting:** Executive-level analysis delivery via QuestPDF
- ✓ **Interactive ER Visualization:** Mermaid.js-based dynamic schema diagrams

2. Project Overview

2.1 Vision & Objectives

The project addresses the critical challenge of database technical debt and hidden architectural vulnerabilities. Traditional database administration often relies on manual inspection and ad-hoc queries, creating blind spots for performance bottlenecks, security weaknesses, and scalability issues.

AI Database Analyzer provides an automated, AI-augmented approach to database auditing that:

1. **Extracts Complete Schema Metadata** — Comprehensive extraction of tables, columns, indexes, triggers, stored procedures, views, and constraints.
2. **Performs Intelligent Analysis** — Leverages LLMs to interpret complex database patterns and provide expert-level recommendations.
3. **Generates Actionable Reports** — Professional PDF reports with structured, prioritized recommendations.
4. **Supports Multiple Deployment Models** — Desktop (MAUI) and mobile/web (Flutter) applications.

2.2 Problem Statement

Database schemas frequently become “black boxes” due to:

- **Technical Debt Accumulation:** Organic growth without systematic refactoring.
- **Hidden Performance Bottlenecks:** Non-sargable queries, missing indexes, over-indexing.
- **Security Vulnerabilities:** PII/PHI data exposure, weak access controls, insufficient audit trails.
- **Normalization Violations:** Data redundancy, consistency risks, update anomalies.
- **Scalability Constraints:** Improper partitioning, monolithic designs unsuitable for microservices.

2.3 Solution Approach

The system combines:

- **Deterministic Schema Extraction:** Multi-engine SQL queries for accurate meta-

data collection.

- **AI-Powered Analysis:** LLM prompts designed by database experts covering 30+ audit scenarios.
- **Human-in-the-Loop:** Visual schema exploration, topic selection, and custom analysis prompts.
- **Professional Delivery:** Interactive visualizations and comprehensive PDF reports.

3. System Architecture

3.1 High-Level Architecture Overview

The system follows a layered, service-oriented architecture connecting two client platforms (MAUI and Flutter) through a shared application and data-processing stack to external AI services and target databases.

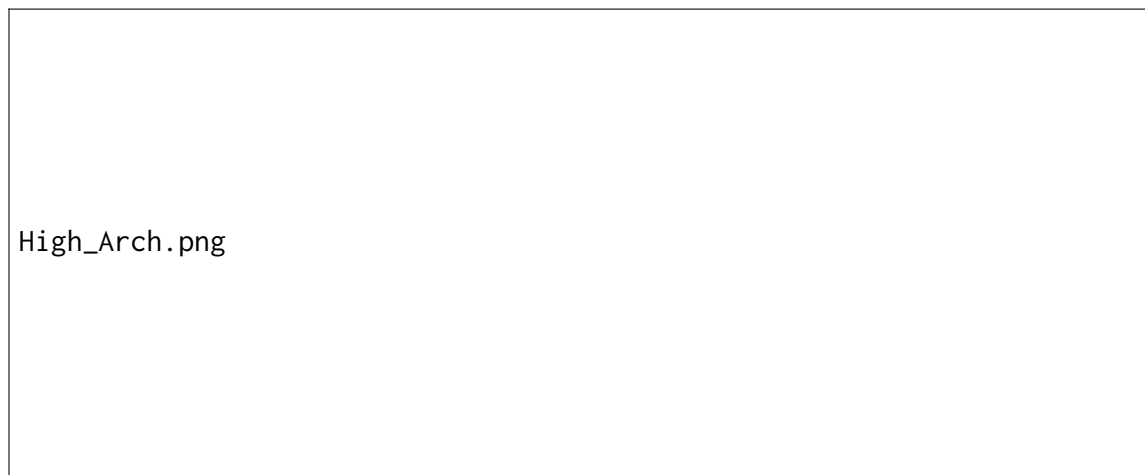


Figure 3.1: High-Level Architecture Overview — Client Applications, Application Layer, Data Processing, External Services, Databases, and Output Generation.

3.2 Layered Architecture

The application is divided into four horizontal layers, each with clearly defined responsibilities and interfaces, as shown below.

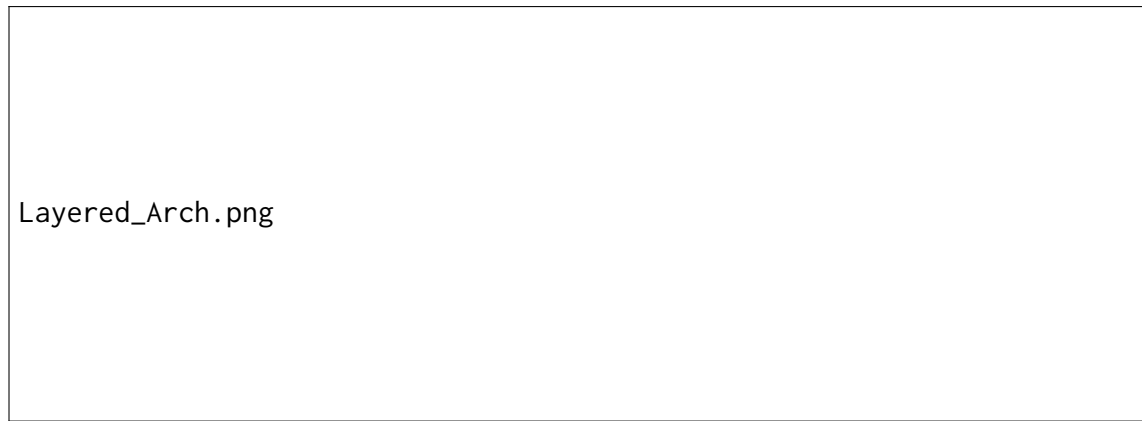


Figure 3.2: Layered Architecture — Presentation, ViewModel/Provider, Service, and Data Access Layers with External AI Integrations.

3.2.1 Layer Descriptions

- **Presentation Layer:** MAUI XAML/C# components and Flutter Dart widgets.
- **ViewModel/Provider Layer:** MainViewModel, SettingsViewModel, VisualizerViewModel (MAUI); MainProvider, SettingsProvider (Flutter).
- **Service Layer:** IDbSchemaService, ILlmService, PdfReportService — shared business logic.
- **Data Access Layer:** Multi-engine connector supporting SQL Server, PostgreSQL, and MySQL.

4. Technology Stack

4.1 Backend & Core Services

Table 4.1: Backend and Core Services

Technology	Version	Purpose
.NET	9.0	Framework for MAUI application
C#	Modern	MAUI backend implementation
MVVM Toolkit	Latest	Observable objects, RelayCommand
ASP.NET Core	9.0	HTTP client services
Dependency Injection	Built-in	Service registration & resolution

4.2 Database Connectivity

Table 4.2: Database Drivers and Support

Database	Driver / Package	Support
SQL Server	Microsoft.Data.SqlClient	Full support (preferred)
PostgreSQL	Npgsql	Full support
MySQL	MySQLConnector	Full support

4.3 Frontend — Desktop (MAUI)

Table 4.3: MAUI Desktop Frontend Components

Component	Technology	Purpose
UI Framework	.NET MAUI	Cross-platform native UI
Layout	XAML	Declarative UI markup
Theming	MAUI Styles	Dark theme implementation
Platform Support	Windows, Android, iOS, macOS	Multi-platform deployment

4.4 Frontend — Mobile/Web (Flutter)

Table 4.4: Flutter Mobile/Web Frontend Components

Component	Version	Purpose
Flutter SDK	3.4+	Cross-platform framework
Dart	3.4+	Programming language
Provider	6.1.2	State management
Material Design	3.0	UI components
WebView	4.10.0	Mermaid diagram rendering
Flutter		

4.5 AI Integration

Table 4.5: AI Integration Services

Service	Type	Purpose	Configuration
Google Gemini	Cloud API	High-scale AI analysis	API key in settings
Ollama	Local	Privacy-first processing	Local HTTP server
HTTP Client	Library	REST communication	Built-in .NET & Flutter

4.6 Reporting & Visualization

Table 4.6: Reporting and Visualization Components

Component	Package	Purpose
QuestPDF	Community Edition	PDF generation & layout
Mermaid.js	Embedded HTML	ER diagram visualization
SVG-Pan-Zoom	JavaScript	Interactive diagram controls

4.7 Core Dependency Graph



Figure 4.1: Core Application Dependency Graph — UI Frameworks, State Management, Database Drivers, HTTP, Reporting, Visualization, and Persistence.

5. Data Flow Analysis

5.1 Connection & Schema Extraction Flow

The following sequence diagram describes how the application establishes a database connection, enumerates databases, and performs multi-result schema extraction in a single round-trip.

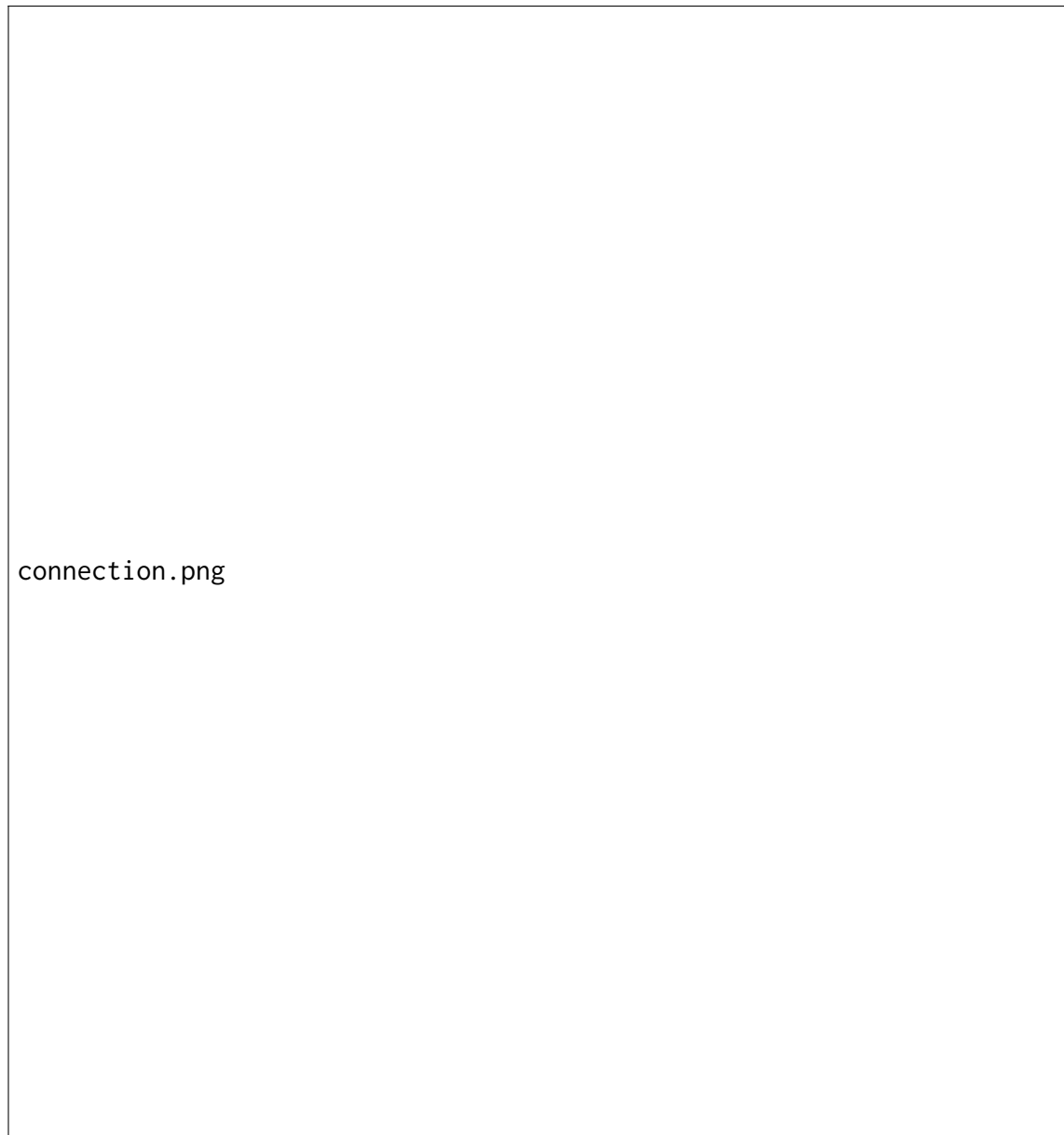


Figure 5.1: Connection & Schema Extraction Sequence Diagram — From user input through multi-result metadata extraction to schema JSON delivery.

5.2 Analysis & Report Generation Flow

Once the schema JSON is available, the analysis pipeline sends each selected topic to the configured AI engine and assembles a structured PDF upon completion.

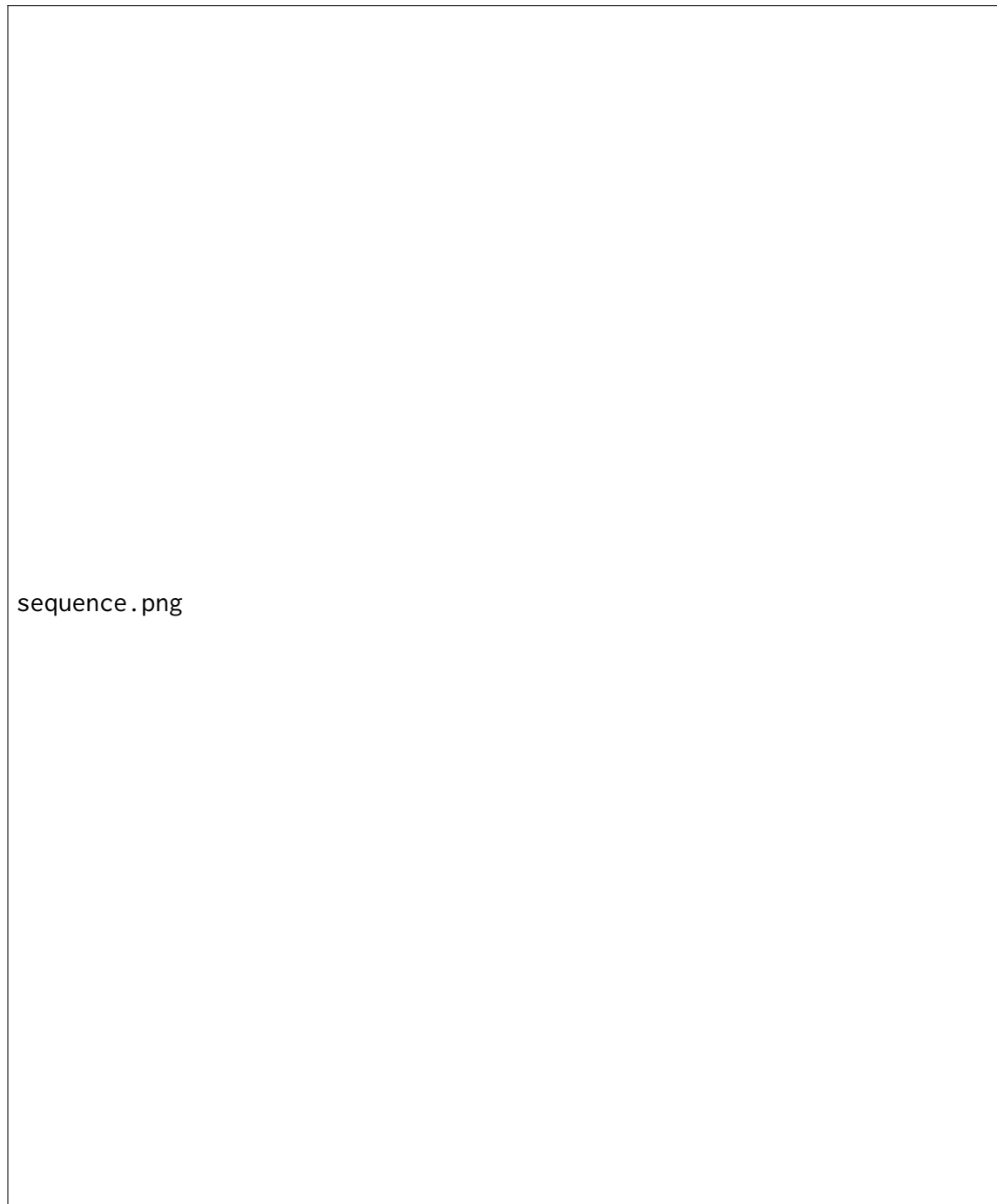


Figure 5.2: Analysis & Report Generation Sequence Diagram — Topic selection, AI processing loop, and PDF assembly.

6. Use Case Diagrams

6.1 Primary User — Database Administrator

The core use cases revolve around the DBA persona: connecting to a database, extracting schema metadata, browsing previews, running AI analyses, generating reports, and configuring settings.



Figure 6.1: Primary Use Case Diagram — DBA/Architect actor with core use cases and their included sub-use cases.

6.2 Extended Use Cases — Analysis Scenarios

Multiple roles interact with the system for different audit scenarios. Each produces a professional report, ER diagram, or actionable insights.

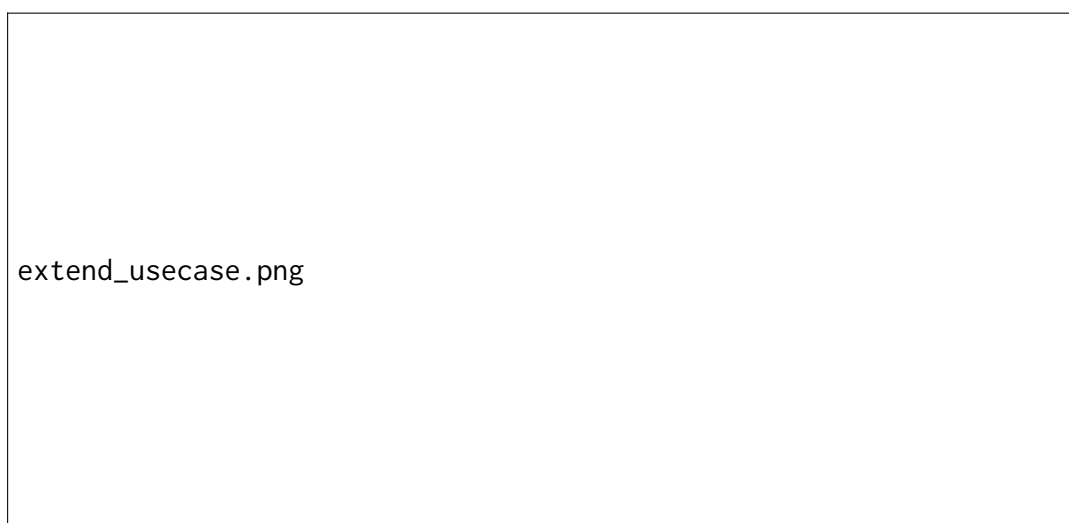


Figure 6.2: Extended Use Case Diagram — DBA, Developer, Data Architect, and Security Officer actors with analysis scenarios and outcomes.

7. Component Architecture

7.1 MAUI Application Component Diagram

The MAUI application follows strict MVVM separation. The **App** shell hosts three pages; each page binds to its dedicated **ViewModel**, which delegates to injected services.

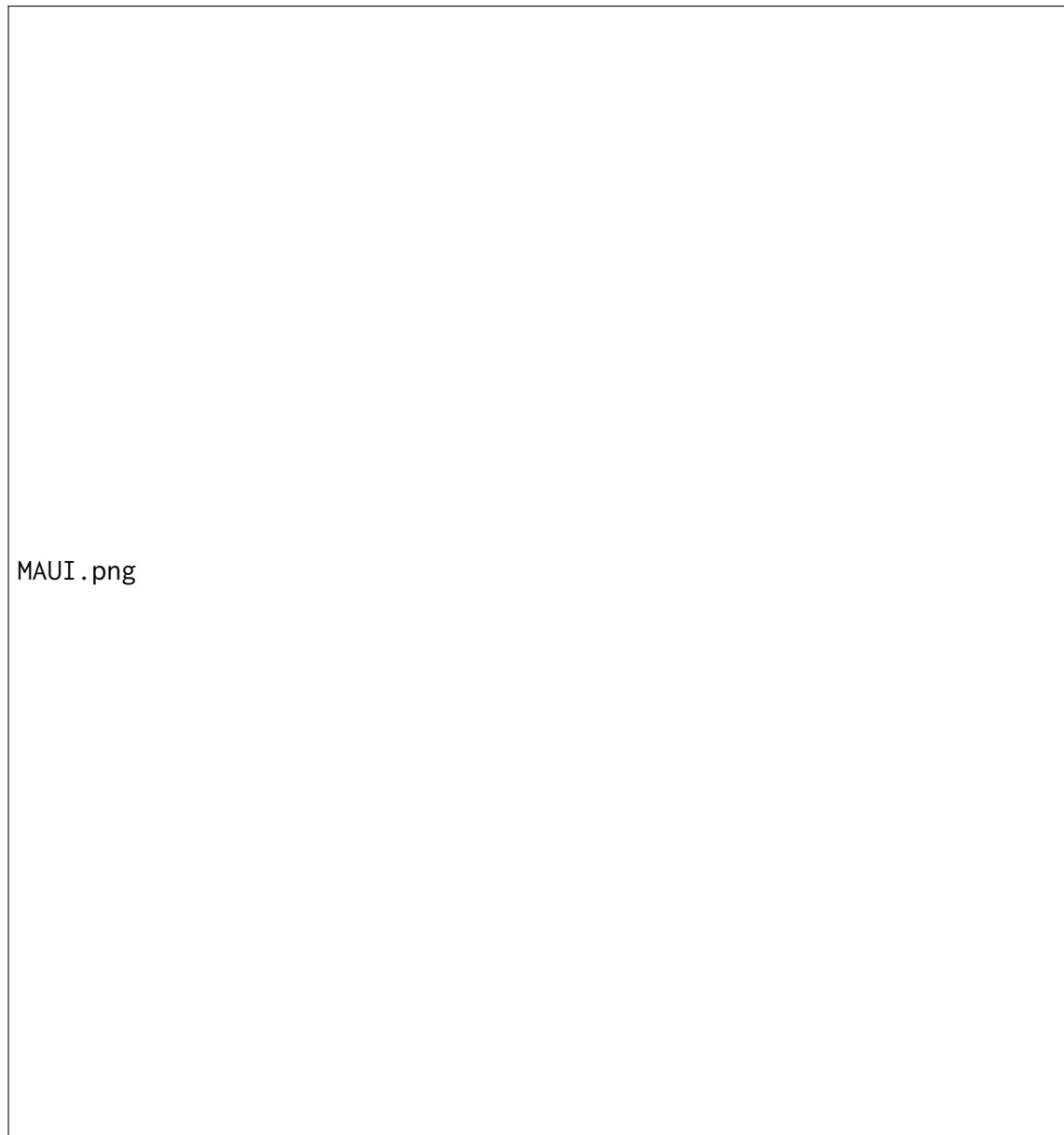


Figure 7.1: MAUI Application Class Diagram — App shell, Pages, ViewModels, DbSchemaService, LlmService interface hierarchy, and PdfReportService.

7.2 Flutter Application Component Diagram

The Flutter counterpart uses `ChangeNotifier`-based Providers as the equivalent of ViewModels, with the same service contracts shared across platforms.

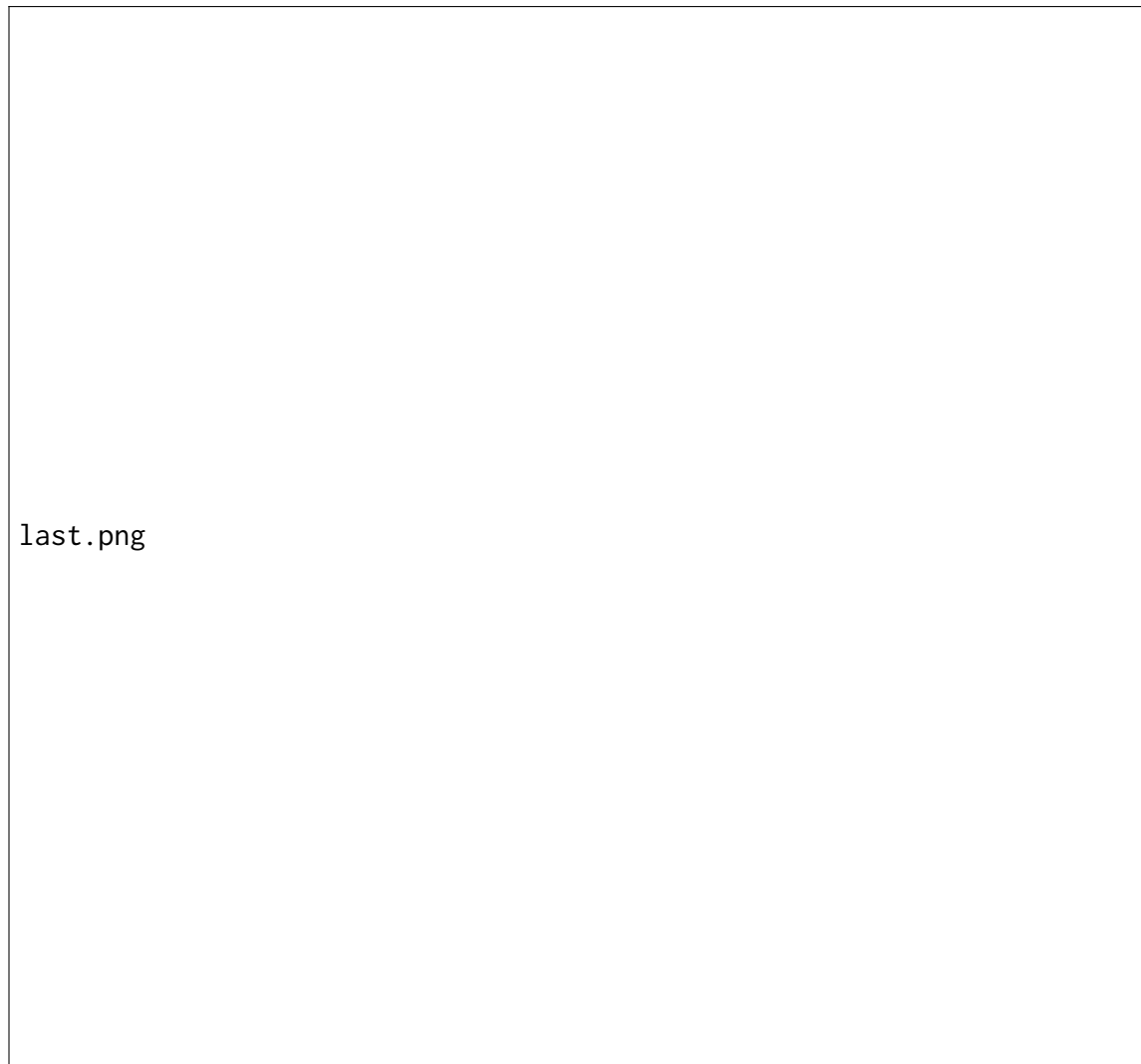


Figure 7.2: Flutter Application Class Diagram — App, Screens, MainProvider
ChangeNotifier, DbSchemaService, LlmService, and PdfReportService.

8. Key Features & Capabilities

8.1 Multi-Database Connectivity

Supported Engines: SQL Server (primary — auto-discovery on Windows), PostgreSQL (full schema extraction), MySQL (complete support).

Connection Features:

- Secure credential storage (Windows Credential Manager / SecureStorage)
- Connection pooling & timeout management
- Test connection validation
- Automatic database enumeration

Metadata Extraction Scope:

- Tables with full column definitions
- Indexes (unique, composite, clustered)
- Foreign key relationships
- Triggers & stored procedures
- Views & computed columns
- Sample data retrieval (3 random rows per table)

8.2 Hybrid AI Integration

8.2.1 Google Gemini (Cloud)

- High-scale processing for large schemas
- Advanced reasoning on complex patterns
- Requires API key (managed in Settings)
- Pay-as-you-go pricing model

8.2.2 Ollama (Local)

- Privacy-first, on-device processing
- Zero API dependency
- Supports various LLM models (Llama, Mistral, Neural Chat)

- Configurable via Settings (default: `localhost:11434`)

8.2.3 Smart Toggling

- User selects preferred provider before analysis
- Same prompts work with both engines
- Fallback capability if one provider unavailable

8.3 Professional Reporting

PDF Generation (via QuestPDF):

- Executive-level formatting
- Multi-page capabilities
- Structured sections for each analysis topic
- Page numbering & headers
- Professional typography

Report Contents:

- Database name & connection info
- Schema statistics (table count, column count, relationship count)
- Analysis results for each selected topic
- Recommendations structured by priority
- Visual dividers & formatting

8.4 Interactive Schema Visualization

Mermaid ER Diagrams:

- Entity-Relationship diagrams auto-generated from extracted schema
- Tables, columns, and relationships visualized
- Zoom & pan controls (SVG-Pan-Zoom)
- Click-to-explore functionality
- Export-ready format

Schema Preview:

- Tree view of tables & columns
- Column data types displayed
- Index information highlighted
- Foreign key relationships shown

8.5 Expert-Level Analysis Prompts

8.5.1 Performance & Indexing (7 Analyses)

1. Non-SARGable Query Predicates & Missing Indexes
2. Over-indexing & Overlapping Index Costs
3. Trigger-Induced Performance Bottlenecks
4. Execution Plan Simulation
5. Foreign Key Index Deficiency
6. Heap Tables & Primary Key Analysis
7. Index Fragmentation & Maintenance

8.5.2 Schema Design & Data Modeling (13 Analyses)

1. Bloated Records & Data Type Wastage
2. 3NF Normalization Violations
3. Poorly Designed Denormalization
4. Orphan Records Detection
5. Circular Dependency Mapping
6. Hidden Complexity in Views
7. Recursive CTE Misusage
8. Data Type Inconsistency
9. Table Width & Page Fragmentation
10. Identity Column Capacity Audit
11. Naming Standards
12. Automated Data Dictionary
13. Siloed Schema Modules (DDD)

8.5.3 Security & Privacy (8 Analyses)

1. RBAC & Least Privilege Violations
2. PII/PHI Encryption Gaps
3. Client-Side Control Over-Reliance
4. Privilege Escalation (IDOR)
5. Dynamic Procedures & SQL Injection
6. Double Spending & Race Conditions
7. Missing Audit Trails
8. GDPR/HIPAA Retention Mechanisms

8.5.4 Scalability & Architecture (3+ Analyses)

1. Horizontal Scalability Bottlenecks
2. Improper Time-Series Storage
3. ORM Anti-Patterns
4. Microservice Boundary Grouping
5. Platform Migration Risk

8.6 Customizable Analysis

- **Custom Question Feature:** Add ad-hoc questions beyond pre-built scenarios; full schema context included automatically; results integrated into final report.
- **Topic Selection:** Checkbox interface for granular control; quick presets (Security Focus, Performance, Data Design).

9. Database Support

9.1 SQL Server

```
1 -- Multi-result query extracts:
2 -- 1. Tables & Columns (with data types, lengths)
3 -- 2. Foreign Key Relationships
4 -- 3. Indexes (unique/composite flags)
5 -- 4. Triggers (mapped to tables)
6 -- 5. Views (list)
7 -- 6. Stored Procedures
8 -- 7. Constraints (CHECK, DEFAULT)
9 -- 8. Sample Data (TOP 3 per table)
```

Listing 9.1: SQL Server Multi-Result Schema Extraction (Pseudocode)

Connection String Example:

```
Server=DESKTOP-PC\SQLEXPRESS;Database=AdventureWorks;
User Id=sa;Password=123456;
```

Listing 9.2: SQL Server Connection String

9.2 PostgreSQL

```
1 -- Equivalent queries via pg_catalog system tables
2 -- Full DDL extraction
3 -- UDT (User-Defined Type) support
```

Listing 9.3: PostgreSQL Schema Extraction via System Catalogs

Connection String Example:

```
Host=localhost;Port=5432;Database=postgres;
Username=postgres;Password=pass;
```

Listing 9.4: PostgreSQL Connection String

9.3 MySQL

```
1 -- INFORMATION_SCHEMA-based queries
2 -- Storage engine detection
3 -- Character set considerations
```

Listing 9.5: MySQL Schema Extraction via INFORMATION_SCHEMA

Connection String Example:

```
Server=localhost;Port=3306;Database=information_schema;  
User=root;Password=pass;
```

Listing 9.6: MySQL Connection String

10. Implementation Details

10.1 State Management Architecture

10.1.1 MAUI — MVVM Toolkit

```
1 [ObservableProperty] private string serverAddress = "DESKTOP-PC\\  
   SQLEXPRESS";  
2 [ObservableProperty] private bool isBusy;  
3 [ObservableProperty] private string selectedDatabase;  
4  
5 public ObservableCollection<string> Databases { get; } = new();  
6 public ObservableCollection<TableSelection> Tables { get; } = new();
```

Listing 10.1: MAUI Observable Properties and Collections

Benefits:

- Reactive property binding
- Automatic INotifyPropertyChanged implementation
- Observable collections for UI sync
- Relay commands for button actions

10.1.2 Flutter — Provider Pattern

```
1 class MainProvider extends ChangeNotifier {  
2   String serverAddress = 'localhost';  
3   bool _isBusy = false;  
4  
5   void notifyListeners() { /* update UI */ }  
6   void _goTo(WizardStep step) { /* navigation */ }  
7 }
```

Listing 10.2: Flutter ChangeNotifier Provider (MainProvider)

Benefits:

- Simple ChangeNotifier pattern
- Scoped provider contexts
- Compile-time safety with strong typing

10.2 Dependency Injection Setup

```
1 builder.Services.AddHttpClient<ILlmService, LlmService>();
2 builder.Services.AddSingleton<DbSchemaService>();
3 builder.Services.AddSingleton<PdfReportService>();
4 builder.Services.AddTransient<MainViewModel>();
```

Listing 10.3: MAUI Service Registration (MauiProgram.cs)

```
1 final DbSchemaService _dbService = DbSchemaService();
2 final LlmService _llmService = LlmService();
3 final PdfReportService _pdfService = PdfReportService();
```

Listing 10.4: Flutter Service Instantiation (MainProvider)

10.3 Error Handling & Validation

1. **Connection Validation:** Test button before full schema extraction; timeout protection (15 second default); credential verification.
2. **Schema Extraction:** Multi-result reader error handling; null-checking for optional metadata; token limit validation for AI context.
3. **AI Processing:** Timeout for long-running analyses; fallback between providers; response parsing error handling.
4. **File Operations:** PDF generation with exception boundaries; path validation for storage; overwrite prompts.

10.4 Performance Optimizations

1. **Parallel Queries:** Multi-result readers execute all metadata queries in a single round-trip, reducing database connection overhead.
2. **Async/Await Pattern:** Non-blocking UI operations; cancellation token support; UI remains responsive during processing.
3. **Schema Filtering:** Select specific tables before extraction; optional components reduce payload.
4. **JSON Compression:** Efficient schema serialization; context-window optimization for LLM prompts.

11. User Interface & Workflows

11.1 MAUI Desktop Application — Step-by-Step Workflow

Step 1 — Connection Input server address, port, username, password; select database engine (SQL Server / PostgreSQL / MySQL); test connection; output: list of available databases.

Step 2 — DB & Table Selection Select target database; tables auto-loaded; multi-select checkboxes; search/filter tables; select components (views, indexes, triggers, stored procedures).

Step 3 — Schema Preview JSON schema displayed; statistics (# tables, columns, relationships); option to fetch sample data; proceed or go back.

Step 4 — Topic Selection 30+ expert prompts pre-loaded; multi-select topics; custom question input; visual selected indicators.

Step 5 — Analysis Progress Real-time progress display; current topic shown; estimated time remaining; cancel option available.

Step 6 — Report & Export View analysis results; export to PDF; open in default viewer; save location displayed.

11.1.1 Visual Theme

- Dark theme (Slate Blue primary #2563EB)
- Material Design 3
- Google Fonts (Inter typeface)
- Responsive layout

11.2 Flutter Mobile/Web Application

The Flutter application replicates the same six-step wizard with a touch-optimized interface:

- Touch-optimized buttons & inputs
- Responsive grid layouts
- WebView for Mermaid diagram rendering
- Landscape & portrait support
- Bottom navigation for screen switching

Navigation Structure:

```
MainScreen (Primary)
|
|-- settings_screen (Settings)
|   |-- Gemini API key
|   |-- Ollama URL
|   +-- Theme toggle
|
+-- visualizer_screen (Visualizer)
    +-- Mermaid ER Diagram
```

Listing 11.1: Flutter Application Navigation Tree

12. Analysis Domains

12.1 Performance & Indexing Analysis

What It Detects:

- Missing indexes on high-cardinality columns
- Redundant composite indexes
- Unindexed foreign keys
- Heap tables without clustered indexes
- Non-SARGable predicates

+ Example Finding:

“Table ‘Orders’ has foreign key ‘FK_Orders_Customers’ but lacks an index on ‘CustomerID’. This will cause full table scans on cascading deletes impacting performance by ~40%.”

Recommended Actions:

```
1 CREATE INDEX IX_Orders_CustomerID ON Orders(CustomerID);  
2 SET STATISTICS IO ON;
```

Listing 12.1: Performance Fix — Create Missing Index

12.2 Schema Design & Data Modeling

What It Detects:

- 3NF violations (transitive dependencies)
- Data type inconsistencies
- Bloated string columns
- Missing constraints (CHECK, DEFAULT)
- Naming standard violations
- Circular dependencies

+ Example Finding:

“Column ‘IsActive’ stored as VARCHAR(255) containing only ‘Y’/‘N’ values. Could use CHAR(1) or BIT, saving 254 bytes per row.”

```
1 ALTER TABLE Users ALTER COLUMN IsActive BIT;  
2 ADD CONSTRAINT CK_IsActive CHECK (IsActive IN (0, 1));
```

Listing 12.2: Schema Fix — Data Type Correction

12.3 Security & Compliance

What It Detects:

- Unencrypted PII/PHI data (emails, SSNs, health records)
- Missing audit columns (created_by, updated_at)
- Overly permissive access patterns
- GDPR/HIPAA data retention gaps
- SQL injection vulnerabilities in procedures
- Privilege escalation paths

+ Example Finding:

“Table ‘Patients’ lacks encryption on ‘MedicalHistory’ column containing PHI. HIPAA requires encryption at rest and in transit.”

```
1 ALTER DATABASE MyDb SET ENCRYPTION ON;  
2 ADD COLUMN CreatedAt DATETIME DEFAULT GETDATE();
```

Listing 12.3: Security Fix — Enable TDE and Audit Trail

12.4 Scalability & Architecture

What It Detects:

- Monolithic schema unsuitable for sharding
- Time-series data in relational B-Trees
- ORM anti-patterns (N+1 queries)
- Poor partition key candidates
- Microservice boundary violations
- Connection pool exhaustion risks

+ Example Finding:

“Table ‘Logs’ stores 10M+ daily time-series events as rows. Consider moving to column-store (CI) or separate OLAP system (Data Warehouse).”

```
1 PARTITION BY RANGE (LogDate);  
2 CREATE CLUSTERED COLUMNSTORE INDEX CCI_Logs ON Logs;
```

Listing 12.4: Scalability Fix — Partitioning and Columnstore

13. Security & Data Handling

13.1 Credential Management

Windows (MAUI):

- Uses Windows Credential Manager
- Credentials never stored in plain text
- Encrypted via DPAPI (Data Protection API)

Cross-Platform (Flutter):

- SharedPreferences with encryption
- API keys stored securely
- Optional biometric unlock

13.2 Data Privacy

1. **Connection Information:** Only used during active session; not persisted (except `savedCredentials` option); cleared on app exit.
2. **Schema Data:** Extracted schema remains local; only sent to AI service if analysis requested; user controls what is sent.
3. **AI Processing:** Ollama (local) — schema never leaves device; Gemini (cloud) — transmitted securely via HTTPS; no logging/retention of prompts.
4. **PDF Reports:** Generated locally; no automatic cloud upload; user controls distribution.

13.3 Access Controls

Application-Level:

- Settings page controls API keys
- User must enable Gemini before cloud analysis
- Can restrict to Ollama-only operation

Database-Level:

- Requires valid credentials
- Respects database permissions

- Schema extraction scoped to accessible tables

14. Deployment Architecture

14.1 MAUI Desktop Deployment

Table 14.1: MAUI Platform Support Matrix

Platform	Status	Min Version
Windows	✓	10.0.19041.0+
Android	✓	6.0+
iOS	✓	12.0+
macOS (Catalyst)	✓	Supported
Linux	Limited	Build possible, native support limited

```
dotnet publish -f net9.0-windows10.0.19041.0 -c Release
```

Listing 14.1: MAUI Windows Release Build Command

14.2 Flutter Application Deployment

Table 14.2: Flutter Target Platform Matrix

Target	Status	Notes
Web (Flutter Web)	✓	Firebase, Vercel, GitHub Pages
Android (APK / AAB)	✓	Google Play Store
iOS (IPA)	✓	App Store
macOS	✓	Native binary
Linux (GTK)	✓	Native binary
Windows (Win32)	✓	Native binary

```
flutter build web --release
```

Listing 14.2: Flutter Web Release Build Command

14.3 Infrastructure Requirements

Development Environment:

- Visual Studio 2022/2023 (MAUI)
- VS Code + Flutter SDK (Flutter)
- SQL Server / PostgreSQL / MySQL for testing
- Ollama (optional, local AI)

Production Requirements:

- Network access to target databases
- Internet access for Gemini API (if enabled)
- Local Ollama server (if privacy-first mode only)
- 50 MB+ storage for application
- 200 MB+ RAM for schema extraction

15. Testing & Validation

15.1 Unit Testing Strategy

```
1 [TestClass]
2 public class DbSchemaServiceTests
3 {
4     [TestMethod]
5     public async Task
6         ExtractSchemaAsync_WithValidConnection_ReturnsValidJson()
7     {
8         // Arrange
9         var service = new DbSchemaService();
10        var config = new SchemaFilterConfig
11        {
12            SelectedTables = new() { "Users" }
13        };
14
15        // Act
16        var result = await service.ExtractSchemaAsJsonAsync(
17            DbSchemaService.DbEngine.SqlServer,
18            _connectionString,
19            config
20        );
21
22        // Assert
23        Assert.IsNotNull(result);
24        Assert.IsTrue(result.Contains("\"Tables\""));
25        Assert.IsTrue(result.Contains("\"ForeignKeys\""));
26    }
27 }
```

Listing 15.1: Unit Test — DbSchemaService Schema Extraction

15.2 Integration Testing

1. **Database Connection Tests:** SQL Server, PostgreSQL, and MySQL connectivity.
2. **Schema Extraction Tests:** Multiple database types, varying schema complexities, large schema handling (1000+ tables).
3. **AI Integration Tests:** Gemini API communication, Ollama local server, prompt validation, response parsing.
4. **UI/UX Tests:** Workflow completion (connection → analysis → report), error scenarios, state persistence, performance under load.

15.3 Manual Testing Checklist

15.3.1 Connection Phase

- ☐ SQL Server connection successful
- ☐ PostgreSQL connection successful
- ☐ MySQL connection successful
- ☐ Invalid credentials show error
- ☐ Test connection button works
- ☐ Database list populates
- ☐ Selected database switches context

15.3.2 Extraction Phase

- ☐ Table selection works
- ☐ Component toggles function
- ☐ Sample data optional
- ☐ JSON schema generates
- ☐ Schema statistics accurate
- ☐ Large schemas handled (>100 tables)

15.3.3 Analysis Phase

- ☐ Topic selection works
- ☐ Custom question input accepted
- ☐ Gemini analysis completes (with API key)
- ☐ Ollama analysis completes (with server)
- ☐ Progress updates display
- ☐ Results visible after completion

15.3.4 Report Phase

- ☐ PDF generates successfully
- ☐ PDF opens in default viewer
- ☐ File saved to correct location
- ☐ Content formatted properly
- ☐ All analyses included

15.3.5 Settings Phase

- ☐ Gemini API key stored

- ☐ Ollama URL editable
- ☐ Settings persist after restart
- ☐ Model switching works

16. Project Metrics

16.1 Codebase Statistics

Table 16.1: Codebase Statistics by Platform

Metric	MAUI	Flutter	Total
Files	~20	~15	~35
Lines of Code	~2,500	~2,000	~4,500
Services	4	3	7
ViewModels / Providers	3	2	5
Pages / Screens	3	3	6
Pre-Built Analyses	30+	30+	30+

16.2 Features Coverage

Table 16.2: Feature Coverage Summary

Type	Count	Status
Supported Databases	3	✓ Complete
Analysis Topics	30+	✓ Complete
Export Formats	2	✓ PDF, JSON
Visualization Types	2	✓ ER, Stats
Platform Support	8+	✓ Complete
AI Providers	2	✓ Complete

17. Architecture Decision Records (ADRs)

17.1 ADR-001: Dual-Platform Strategy (MAUI + Flutter)

Decision: Maintain separate codebases for desktop (MAUI) and mobile/web (Flutter).

Rationale:

- MAUI provides native desktop experience on Windows with the .NET ecosystem.
- Flutter provides superior mobile experience and web support.
- Shared business logic via services (DbSchemaService, LlmService).
- Different UI paradigms suit different platforms.

Trade-offs:

- Code duplication in view layers (acceptable).
- Twice the maintenance burden (mitigated by service-layer consistency).
- Independent release cycles (beneficial for platform-specific optimizations).

17.2 ADR-002: Hybrid AI Architecture

Decision: Support both cloud (Gemini) and local (Ollama) AI processing.

Rationale:

- Cloud: handles large schemas and advanced reasoning.
- Local: privacy-preserving, offline-capable.
- User agency: choose preferred provider.
- Fallback capability if one service is unavailable.

Trade-offs:

- Increased implementation complexity.
- Different response quality/latency profiles.
- User education required.

17.3 ADR-003: MVVM for State Management

Decision: Use MVVM Toolkit (MAUI) and Provider (Flutter) for reactive state.

Rationale:

- Established pattern in the .NET ecosystem.
- Observable properties enable UI binding.
- Testable business logic separation.
- Handles async operations cleanly.

Trade-offs:

- Learning curve for new developers.
- Boilerplate code in ViewModels.
- Runtime dependency injections.

18. Conclusion

18.1 Project Achievements

AI Database Analyzer successfully delivers on its vision of automated, AI-driven database administration:

- ✓ **Comprehensive Schema Analysis** — Extracts complete metadata from 3 major database engines.
- ✓ **Intelligent Auditing** — 30+ expert-level analysis prompts covering performance, security, and scalability.
- ✓ **Hybrid AI Processing** — Seamless integration with both cloud and local AI models.
- ✓ **Professional Reporting** — Executive-grade PDF reports and interactive visualizations.
- ✓ **Multi-Platform** — Desktop (MAUI) and mobile/web (Flutter) implementations.
- ✓ **Enterprise-Grade** — MVVM architecture, clean code, security-conscious design.
- ✓ **User-Centric** — Intuitive wizard-based workflow, customizable analysis, export capabilities.

18.2 Innovation Highlights

- **AI-DBA Consulting Automation:** Replaces senior DBA consulting for many standard audits.
- **Schema Context Window Optimization:** Efficiently sends large schemas to LLMs within token constraints.
- **Privacy-First Analysis:** Local Ollama option for sensitive environments.
- **Multi-Dialect SQL Support:** Abstracts database differences behind a unified service interface.

18.3 Educational Value (DAM Course)

This project demonstrates advanced database administration concepts:

1. **Metadata Extraction** — Understanding database system catalogs.
2. **Schema Analysis** — Normalization, indexing, referential integrity.

3. **Performance Tuning** — Index strategies, query optimization.
4. **Security Hardening** — Access control, PII protection, audit trails.
5. **Scalability Planning** — Partitioning, microservices boundaries, time-series handling.
6. **Modern Architecture** — Microservices, cloud-native design, AI integration.
7. **Full-Stack Development** — Backend (C# services), Frontend (MAUI, Flutter), Infrastructure.

18.4 Future Enhancement Opportunities

1. **Advanced Visualizations:** Query execution plans as diagrams, performance heat maps, dependency graphs.
2. **Extended Database Support:** Oracle Database, MongoDB (NoSQL), Snowflake/BigQuery (Cloud DWH).
3. **Collaborative Features:** Team reports and sharing, comments/annotations on findings, role-based access.
4. **Performance Monitoring:** Real-time metrics integration, query profiling, anomaly detection.
5. **Remediation Automation:** Generate ALTER scripts, execute recommendations, track changes.

References

Technologies

- .NET MAUI Documentation: <https://learn.microsoft.com/maui/>
- Flutter Documentation: <https://flutter.dev/docs>
- Google Gemini API: <https://ai.google.dev/>
- Ollama: <https://ollama.ai/>
- QuestPDF: <https://www.questpdf.com/>

Database-Related

- SQL Server System Catalog: <https://learn.microsoft.com/sql/relational-databases/system-catalog-views/>
- PostgreSQL System Catalogs: <https://www.postgresql.org/docs/current/catalogs.html>
- MySQL Information Schema: <https://dev.mysql.com/doc/refman/8.0/en/information-schema.html>

Best Practices

- Microsoft .NET Architecture: <https://learn.microsoft.com/dotnet/architecture/>
- Clean Architecture Principles: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
- Project Link: https://github.com/ahlat2004/DBAM_Project

A. Sample Configuration Files

A.1 MAUI Dependency Injection Configuration

```
1 public static class MauiProgram
2 {
3     public static MauiApp CreateMauiApp()
4     {
5         var builder = MauiApp.CreateBuilder();
6         builder
7             .UseMauiApp<App>()
8             .ConfigureFonts(fonts =>
9             {
10                 fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
11             });
12
13         builder.Services.AddHttpClient<ILlmService, LlmService>();
14         builder.Services.AddSingleton<DbSchemaService>();
15         builder.Services.AddSingleton<PdfReportService>();
16         builder.Services.AddTransient<MainViewModel>();
17         builder.Services.AddTransient<SettingsViewModel>();
18
19         return builder.Build();
20     }
21 }
```

Listing A.1: MauiProgram.cs — Full DI Setup

A.2 Flutter pubspec.yaml

```
name: ai_database_analyzer
version: 1.0.0+1

dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.2
  http: ^1.2.2
  mssql_connection: ^3.0.0
  pdf: ^3.11.0
  webview_flutter: ^4.10.0

dev_dependencies:
  flutter_test:
    sdk: flutter
```

Listing A.2: pubspec.yaml — Flutter Dependencies

Report Generated: May 17, 2026 · Project Status:  Fully Functional · Next Phase:
Presentation & Defense (May 19–21, 2026)